



Rapport SAE 1.02

1. Introduction :

a. Présentation du sujet :

Le projet consiste à créer des robots joueurs de Puissance 4 qui s'affronteront. L'objectif est d'évaluer leurs performances en termes de niveau de jeu, de stratégie, et de temps de calcul.

Pour ce faire, des scripts Python sont fournis, incluant un moteur de jeu, des exemples de robots, et un outil de configuration. Les robots doivent être capables de choisir où placer leurs pions dans le jeu en fonction de différentes stratégies.

Le moteur de jeu fonctionne en demandant aux robots de jouer tour à tour. Le dictionnaire de la partie transmis aux robots contient des informations sur l'état du jeu, les joueurs, et le dernier coup joué.

Le travail demandé comprend la création d'un robot "naïf" effectuant des choix aléatoires, et d'un robot "intelligent" implémentant une stratégie définie. Une comparaison des performances entre les robots, y compris un robot humain, est également prévue.

Pour évaluer l'efficacité d'une stratégie, il est nécessaire de développer une métrique quantitative. De plus, le temps de calcul de chaque robot sera mesuré sur dix parties pour analyser son évolution en fonction du nombre de coups joués.

b. Présentation des objectifs :

Ce projet a pour objectif principal de concevoir des robots joueurs de Puissance 4 et d'évaluer leurs performances. Les robots seront développés avec des stratégies distinctes, dont un robot "naïf" effectuant des choix aléatoires et un robot "intelligent" avec une stratégie élaborée.

L'évaluation du niveau de jeu se fera par des confrontations entre les robots, incluant également un joueur humain. Une métrique quantitative sera définie pour mesurer l'efficacité des stratégies, et le temps de calcul de chaque robot sera analysé sur dix parties.

Le projet culminera avec la rédaction d'un rapport détaillé et une présentation orale, visant à développer des compétences en programmation, en algorithmique, et à favoriser la créativité dans la conception de stratégies de jeu.

2. Présentation des attendus :

Les attentes pour ce projet comprennent la conception réussie de robots joueurs de Puissance 4, chacun implémentant une stratégie distincte, notamment un robot "naïf" et un robot "intelligent".

Les performances de ces robots seront évaluées à travers des confrontations mutuelles et avec un joueur humain, avec une attention particulière à la définition et à l'application d'une métrique quantitative pour mesurer l'efficacité des stratégies.

Les résultats des comparaisons, y compris le temps de calcul sur dix parties, doivent être consignés dans un rapport détaillé. Les compétences en programmation, en algorithmique et la capacité à présenter de manière claire et concise seront évaluées lors de la présentation orale finale.

L'objectif global est de fournir une compréhension approfondie du processus de création de robots joueurs et de développer des compétences pratiques dans le domaine.

3. Benchmark des solutions :

a. Descriptions des méthodes auxquelles vous avez pensées ou que vous avez trouvées :

J'ai pensé à une méthode pour le robot intelligent dans laquelle je joue dans la colonne n°3 pour commencer, ensuite dans les colonnes n°1 et n°5, et de jouer dans les colonnes n°2 et n°4 pour pouvoir faire une diagonal

gauche vers le haut ou diagonal droite vers le haut. Ce qui pourrait donner l'avantage stratégique afin d'avoir plusieurs possibilités d'alignement afin de contenir l'adversaire et de m'offrir plusieurs possibilités de gagner et que le joueur adverse ne puisse pas contrer toutes les possibilités de d'alignement de 4 pions.

Une autre méthode pour le robot intelligent que j'ai pensée était de jouer dans la colonne n°3 pour commencer, ensuite dans les colonnes n°1 et n°5, et de jouer dans les colonnes n°2 et n°4 pour pouvoir faire une diagonal gauche vers le haut ou diagonal droite vers le haut. Puis de bloquer l'adversaire quand il a 2 pions d'aligner ou 3 pions d'aligner afin de réduire ces chances de gagner, afin que mes pions qui bloque les pions adverses viennent créer des possibilités d'alignement afin que je gagne.

La méthode pour le robot naïf est qu'il joue qu'en aléatoire tous au long de la partie.

b. Comparaison entre les méthodes :

La première méthode pour le robot intelligent peut être bonne mais le problème pour créer des possibilités d'alignement c'est de parcourir toutes les possibilités de la grille afin de jouer le meilleur coup à chaque tour mais il faudrait coder un algorithme MinMax, mais cela dépasse mes capacités.

La seconde méthode semble plus simple à coder mais le problème c'est que en ne faisant que de bloquer il y a de fortes possibilités qu'il y est souvent des égalités ou que le joueur adverse trouve le bon coup à chaque tour.

Pour la méthode du robot naïf cela semble être la meilleure méthode possible pour ce type de robot.

4. Explication de votre solution :

a. Description textuelle des méthodes implémentées :

Ma méthode pour le robot intelligent, c'est que j'ai commencé par jouer dans la colonne n°3, pour avoir l'avantage car je dominerai le centre donc plus de possibilité d'avoir des alignements. Ensuite je joue dans la colonne n°1 ou la colonne n°5 afin de créer une possibilité de diagonales vers le haut à droite ou à gauche.

```
# Choisi de remplir la colonne 3 pour avoir l'avantage du début
if (p["tourDe"] == 1 and p["grille"][2][3] == 0) or (p["tourDe"] == 2 and p["grille"][2][3] == 0):
    return 3

#Place les pion dans la colonne 1 ou 5 après avoir rempli la colonne 3 pour un début de stratégie
if p["grille"][2][3] != 0:
    for col in [1,5]:
        if p["grille"][0][col] == 0 and p["grille"][1][col] == 0:
            return col

# Place les pions dans les colonne 2 et 4 pour former un début de diagonal
for col in [2,4]:
    if p["grille"][0][col] == 0 and p["grille"][1][col - 1] == 0:
        return col
```

Ensuite, je commence par bloquer l'adversaire en bloquant ses pions quand il en a 2 d'aligner dans n'importe quel direction (horizontale, verticale, diagonales : haut, bas, gauche droite) :

```
# Bloque l'adversaire s'il y a 2 pions alignés horizontalement :
if (p["tourDe"] == 1 and p["nbPion"] == 18 ) or (p["tourDe"] == 2 and p["nbPion"] == 18 ):
    for i in range(5):
        for j in range(2,4):
            if ((p["grille"][i][j] == p["tourDe"] and p["grille"][i][j + 1] == p["tourDe"] and p["grille"][i][j + 2] == p["tourDe"]) or (j >= 2 and p["grille"][i][j] == p["tourDe"] and p["grille"][i][j - 1] == p["tourDe"] and p["grille"][i][j - 2] == p["tourDe"])) :
                if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                    return j + 2
                elif p["grille"][5][j] != 0:
                    return random.randint(0,6)

# Bloque l'adversaire s'il y a 2 pions alignés verticalement:
for i in range(3):
    for j in range(5):
        if (p["grille"][i][j] == p["tourDe"] and p["grille"][i+1][j] == p["tourDe"] and p["grille"][i+2][j] == p["tourDe"]):
            if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                return j + 2
            elif p["grille"][5][j] != 0:
                return random.randint(0,6)

# Bloque l'adversaire s'il y a 2 pions alignés diagonalement vers le bas à droite:
for i in range(3):
    for j in range(4):
        if ((p["grille"][i][j] == p["tourDe"] and p["grille"][i+1][j+1] == p["tourDe"] and p["grille"][i+2][j+2] == p["tourDe"]) or (i >= 2 and j >= 2 and p["grille"][i][j] == p["tourDe"] and p["grille"][i-1][j-1] == p["tourDe"] and p["grille"][i-2][j-2] == p["tourDe"])) :
            if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                return j+2
            elif p["grille"][5][j] != 0:
                return random.randint(0,6)

# Bloque l'adversaire s'il y a 2 pions alignés diagonalement vers le bas à gauche:
for i in range(3):
    for j in range(2, 4):
        if ((p["grille"][i][j] == p["tourDe"] and p["grille"][i+1][j-1] == p["tourDe"] and p["grille"][i+2][j-2] == p["tourDe"]) or (p["grille"][i][j] == p["tourDe"] and p["grille"][i-1][j+1] == p["tourDe"] and p["grille"][i-2][j+2] == p["tourDe"])) :
            if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                return j
            elif p["grille"][5][j] != 0:
                return random.randint(0,6)
```

Ensuite, je bloque l'adversaire en bloquant ses pions quand il en a 3 d'aligner dans n'importe quel direction (horizontale, verticale, diagonales : haut, bas, gauche, droite) car l'adversaire va chercher à aligner 3 pions afin d'en aligner 4 par la suite donc je lui bloque ses possibilités :

```
# Bloque l'adversaire s'il y a 3 pions alignés horizontalement:
for i in range(5):
    for j in range(3):
        if ((p["grille"][i][j] == p["tourDe"] and p["grille"][i][j + 1] == p["tourDe"] and p["grille"][i][j + 2] == p["tourDe"] and p["grille"][i][j+3] == p["tourDe"]) or (p["grille"][i][j] == p["tourDe"] and p["grille"][i][j - 1] == p["tourDe"] and p["grille"][i][j - 2] == p["tourDe"] and p["grille"][i][j-3] == p["tourDe"])) :
            if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                return j + 3
            elif p["grille"][5][j] != 0:
                return random.randint(0,6)

# Bloque l'adversaire s'il y a 3 pions alignés verticalement:
for i in range(2):
    for j in range(6):
        if (p["grille"][i][j] == p["tourDe"] and p["grille"][i+1][j] == p["tourDe"] and p["grille"][i+2][j] == p["tourDe"] and p["grille"][i+3][j] == p["tourDe"]) :
            if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                return j+1
            elif p["grille"][5][j] != 0:
                return random.randint(0,6)

# Bloque l'adversaire s'il y a 3 pions alignés diagonalement vers le bas à droite:
for i in range(3):
    for j in range(3):
        if ((p["grille"][i][j] == p["tourDe"] and p["grille"][i+1][j+1] == p["tourDe"] and p["grille"][i+2][j+2] == p["tourDe"] and p["grille"][i+3][j+3] == p["tourDe"]) or (p["grille"][i][j] == p["tourDe"] and p["grille"][i-1][j-1] == p["tourDe"] and p["grille"][i-2][j-2] == p["tourDe"] and p["grille"][i-3][j-3] == p["tourDe"])) :
            if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                return j + 3
            elif p["grille"][5][j] != 0:
                return random.randint(0,6)

# Bloque l'adversaire s'il y a 3 pions alignés diagonalement vers le bas à gauche:
for i in range(2):
    for j in range(3):
        if ((p["grille"][i][j] == p["tourDe"] and p["grille"][i+1][j-1] == p["tourDe"] and p["grille"][i+2][j-2] == p["tourDe"] and p["grille"][i+3][j-3] == 0) or (p["grille"][i][j] == p["tourDe"] and p["grille"][i-1][j+1] == p["tourDe"] and p["grille"][i-2][j+2] == p["tourDe"] and p["grille"][i-3][j+3] == p["tourDe"])) :
            if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                return j + 2
            elif p["grille"][5][j] != 0:
                return random.randint(0,6)
```

Puis, pour mon attaque je commence par aligner 3 pions si j'ai 2 pions d'aligner, afin d'avoir des alignements et en faire plusieurs afin de créer plusieurs possibilités dans toutes les directions possible direction (horizontale, verticale, diagonales : haut, bas, gauche, droite) pour jouer ensuite :

```
#Placement des pions si 2 pion sont alignés:
#horizontalement
for i in range(5):
    for j in range(4):
        if ((p["grille"][i][j] == p["tourDe"] and p["grille"][i][j+1] == p["tourDe"] and p["grille"][i][j+2] == p["tourDe"]) or (p["grille"][i][j] == p["tourDe"] and p["grille"][i][j-1] == p["tourDe"] and p["grille"][i][j-2] == p["tourDe"])) :
            if j+3 < 6 and p["grille"][i][j+3] == 0:
                p["grille"][i][j+3] = p["tourDe"]
                if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                    return j +3
            elif p["grille"][5][j] != 0:
                return random.randint(0,6)

#vérifie Verticalement
for i in range(3):
    for j in range(4):
        if p["grille"][i][j] == p["tourDe"] and p["grille"][i+1][j] == p["tourDe"] and p["grille"][i+2][j] == p["tourDe"] :
            if i-1 >= 0 and p["grille"][i-1][j] == 0:
                p["grille"][i-1][j] = p["tourDe"]
                if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                    return j
            elif p["grille"][5][j] != 0:
                return random.randint(0,6)

#vérifie en diagonales vers le bas à droite
if j+2 >= 6 and ((p["grille"][i][j] == p["tourDe"] and p["grille"][i+1][j+1] == p["tourDe"] and p["grille"][i+2][j+2] == p["tourDe"]) or (p["grille"][i][j] == p["tourDe"] and p["grille"][i-1][j-1] == p["tourDe"] and p["grille"][i-2][j-2] == p["tourDe"])):
    if i-1 >= 0 and j-1 >= 0 and p["grille"][i-1][j-1] == 0:
        p["grille"][i-1][j-1] = p["tourDe"]
        if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
            return j - 1
    elif p["grille"][5][j] != 0:
        return random.randint(0,6)

#vérifie en diagonales vers le bas à gauche
if j-2 >= 0 and ((p["grille"][i][j] == p["tourDe"] and p["grille"][i+1][j-1] == p["tourDe"] and p["grille"][i+2][j-2] == p["tourDe"]) or (p["grille"][i][j] == p["tourDe"] and p["grille"][i-1][j+1] == p["tourDe"] and p["grille"][i-2][j+2] == p["tourDe"])) :
    if i-1 >= 0 and j+1 < 6 and p["grille"][i-1][j+1] == 0:
        p["grille"][i-1][j+1] = p["tourDe"]
        if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
            return j + 1
    elif p["grille"][5][j] != 0:
        return random.randint(0,6)
```

Et, pour finir mon attaque je place un 4^{ème} pion afin d'avoir un alignement de 4 pions et de gagner la partie dans toutes les directions possible direction (horizontale, verticale, diagonales : haut, bas, gauche, droite) pour jouer et en dernier recours je fais un return Radom si je ne peux jouer stratégiquement nulle part :

```
#Placement des pions si 3 pion sont alignés
#Vérification des alignement:
for i in range(5):
    for j in range(6):
        for align in [2,3]:
            # alignement horizontal
            rangEnd = i + (align-1)
            if rangEnd < 5:
                alignment = all (p["grille"][i+k][j] == p["tourDe"] for k in range (align))
                if alignment and (j+ align) < 6 and ((p["grille"][rangEnd + 1][j + align - 1] == p["tourDe"] ) or (p["grille"][rangEnd - 1][j - align + 1] == p["tourDe"] )) :
                    p["grille"][rangEnd + 1][j + align - 1] = p["tourDe"]
                    if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                        return j
                    elif p["grille"][5][j] != 0 :
                        return random.randint(0,6)

            #alignement verticale
            colEnd = j + (align -1)
            if colEnd < 6 :
                alignment = all (p["grille"][i][j+k] == p["tourDe"] for k in range (align))
                if alignment and (i+ align) < 5 and p["grille"][i + align][colEnd + 1] == p["tourDe"] :
                    p["grille"][i + align][colEnd + 1] = p["tourDe"]
                    if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                        return colEnd + 1
                    elif p["grille"][5][j] != 0:
                        return random.randint(0,6)

            #alignement diagonaux bas à droite
            rangEnd = i + (align -1)
            colEnd = j + (align -1)
            if rangEnd < 5 and colEnd < 6 :
                alignment = all(p["grille"][i+k][j+k] == p["tourDe"] for k in range (align))
                if alignment and (j+ align) < 6 and ((p["grille"][rangEnd + 1][colEnd + 1] == p["tourDe"]) or (p["grille"][rangEnd - 1][colEnd - 1] == 0)) :
                    p["grille"][rangEnd + 1][colEnd + 1] = p["tourDe"]
                    if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
                        return colEnd + 1
                    elif p["grille"][5][j] != 0:
                        return random.randint(0,6)
```

```
#alignement diagonaux bas à gauche

rangEnd = i + (align - 1)
colStart = j - (align - 1)
if rangEnd < 5 and colStart >= 0 :
    alignment = all (p["grille"][i+k][j-k] == p["tourDe"] for k in range(align))
    if alignment and (j - 1) >= 0 and ((p["grille"][rangEnd + 1][colStart - 1] == p["tourDe"]) or (p["grille"][rangEnd - 1][colStart + 1] == p["tourDe"])):
        p["grille"][rangEnd + 1][colStart - 1] = p["tourDe"]
        if (0 <= j and j >= 6 and 0 <= i and i >= 5) and p["grille"][5][j] == 0:
            return colStart + 1
        elif p["grille"][5][j] != 0:
            return random.randint(0,6)
return random.randint(0,6)
```

Pour la méthode du robot naïf, j'ai importé la bibliothèque Random afin de faire un « return random.randint(0,6) » afin qu'il joue dans la limite de la grille.

```
import random
def choix(p):
    return random.randint(0,6)
```

b. Adaptation aux structures de données imposées par le sujet :

Pour l'adaptation aux structures imposées j'ai utilisé les dictionnaires proposés dans le sujet afin de rendre mon code plus optimal et plus facile à lire :

- p["grille"] = pour jouer avec la matrice imposée.
- p["tourDe"] = pour déterminer à chaque tour quel joueur joue .
- p[1]["nbPion"] et p[2]["nbPion"] = pour savoir le nombre de pions qui reste aux joueurs ou pour déterminer une condition à partir du nombre de pions qu'il reste.

c. Choix des tests :

Pour les choix des tests j'ai testé mon robot intelligent vs robot naïf , la plupart du temps c'est le robot intelligent qui gagne mais vu qu'il n'est pas parfait parfois le robot naïf gagne.

```
au tour de 01 de j
IA1
0.0
5
horizontal : 4
0123456
XXOOO.O
OOXXXXX
XOXOXOX
OOXXOXO
OXOXOXO
XXOXOXO
```

Comme on peut le voir c'est le robot intelligent qui a gagné mais de justesse car mon robot n'est pas parfait.

Puis j'ai testé mon robot intelligent vs humain, c'est l'humain qui gagne 1 fois sur 2, car mon robot intelligent n'est pas au point.

d. Comparaison des performances et stratégies :

Pour la comparaison des performances et stratégies j'ai modifié le fichier puissance4moteur afin de savoir le temps que met mon robot intelligent en moyenne à jouer pour chaque tour .

Pour faire ces comparaisons j'ai commencé par faire jouer 10 parties mon robot intelligent vs mon robot naïf, avec 5 fois mon robot intelligent en joueur 1 et 5 fois en joueur 2 :

Partie	Joueur 1 : Robot intelligent (en seconde)	Joueur 2 : Robot Naïf (en seconde)	Vainqueur
1	0.10758450627326965	0.0863950252532959	Joueur 1
2	0.09848748313056098	0.07542343139648437	Joueur 1
3	0.08931867281595866	0.09929365581936306	Joueur 2
4	0.0948789914449056	0.0908726692199707	Joueur 1
5	0.06873175501823425	0.08410388628641764	Joueur 1

Partie	Joueur 1 : Robot Naïf (en seconde)	Joueur 2 : Robot Intelligent (en seconde)	Vainqueur
1	0.08616738319396973	0.07588653564453125	Joueur 2
2	0.09676774342854817	0.08905017375946045	Joueur 1
3	0.09343254566192627	0.08628079295158386	Joueur 2
4	0.10118428866068523	0.09120484193166097	Joueur 2
5	0.09442152579625447	0.09285847345987956	Joueur 2

On peut voir que le robot intelligent suit une stratégie et gagne plus souvent que le robot naïf .

Ensuite j'ai fait jouer 10 parties mon robot intelligent vs robot intelligent, avec 5 fois mon robot intelligent en joueur 1 et 5 fois en joueur 2 :

Partie	Joueur 1 : Robot Intelligent (en seconde)	Joueur 2 : Robot Intelligent (en seconde)	Vainqueurs
1	0.16825687885284424	0.13776166439056398	Joueur 2
2	0.12750408053398132	0.12371940082973903	Joueur 1
3	0.1282933235168457	0.12609251340230307	Joueur 1
4	0.11188406693307977	0.1179757429205853	Joueur 1
5	0.1352207443930886	0.12433137496312459	Joueur 1

Partie	Joueur 1 : Robot Intelligent (en seconde)	Joueur 2 : Robot Intelligent (en seconde)	Vainqueurs
1	0.14242947101593018	0.13757166266441345	Joueur 1
2	0.13437116146087646	0.11904582977294922	Joueur 1
3	0.15010249614715576	0.1210148675101144	Joueur 1
4	0.160648783047994	0.14932672182718912	Joueur 2
5	0.15089726448059082	0.13335653146107992	Joueur 1

On peut voir que on a plus d'avantage de gagner quand on est joueur 1 que joueur 2.

Ensuite j'ai fait jouer 10 parties mon robot intelligent vs humain, avec 5 fois mon robot intelligent en joueur 1 et 5 fois en joueur 2 :

Partie	Joueur 1 : Robot Intelligent (en seconde)	Joueur 2 : Humain (en seconde)	Vainqueurs
1	0.09313362836837769	3.536671996116638	Joueur 2
2	0.08487634658813477	3.3875176270802814	Joueur 1
3	0.07861226797103882	3.0443748994307085	Joueur 1
4	0.09733283519744873	2.1665013432502747	Joueur 2
5	0.09733283519744873	2.702546755472819	Joueur 1

Partie	Joueur 1 : Humain (en seconde)	Joueur 2 : Robot Intelligent (en seconde)	Vainqueurs
1	4.195546269416809	0.09170365333557129	Joueur 2
2	4.195546269416809	0.06992673873901367	Joueur 2
3	3.812112013498942	0.06992673873901367	Joueur 1
4	4.724709939956665	0.07284037272135417	Joueur 1

5	5.438920259475708	0.07284037272135417	Joueur 1
---	-------------------	---------------------	----------

Nous pouvons voir que l'humain met plus temps à réfléchir pour jouer que le robot.

Partie	Joueur 1 : Robot Naïf (en seconde)	Joueur 2 : Humain (en seconde)	Vainqueurs
1	0.05689680576324463	2.0873019695281982	Joueur 2
2	0.06004484494527181	1.3972086906433105	Joueur 2
3	0.06034484494527181	1.8755335807800293	Joueur 2
4	0.06372249126434326	2.031610071659088	Joueur 2
5	0.06588239669799804	2.886084032058716	Joueur 2

Partie	Joueur 1 : Humain (en seconde)	Joueur 2 : Robot Intelligent (en seconde)	Vainqueurs
1	5.132948057992118	0.08149131139119466	Joueur 1
2	3.3432201544443765	0.08149131139119466	Joueur 1
3	3.3432201544443765	0.08456168174743653	Joueur 1
4	3.4559107303619383	0.08511954545974731	Joueur 1
5	3.6361820697784424	0.0859667460123698	Joueur 1

Nous pouvons voir que c'est toujours l'humain qui gagne car le robot naïf joue aléatoirement donc n'a pas de stratégie comparé à l'humain.

5. Analyse critique du travail :

a. Points atteints et non atteints :

Durant ce projet les points atteints ont été de faire un robot intelligent , un robot naïf, de faire des comparaisons de performance et de stratégie, mais les points non atteints ont été de faire du robot intelligent un robot qui gagne à tous les coups.

b. Analyse sur l'organisation du travail en groupe :

Selon moi pour un projet comme celui-ci ça a été bien de travailler seul car j'ai pu mettre au point ma stratégie comme je le voulais et rapidement au lieu de discuter et de se mettre d'accord avec les autres membres du groupe donc cela a pu éviter des conflits mais aussi de compter sur soi-même et pas sur des membres du groupe.

6. Bilan global du projet :

Ce projet a été un projet intéressant sur le plan créatif qu'algorithmique, j'ai atteint les objectifs fixés en début de projet, cela a été bénéfique de travailler tout seul car j'ai pu faire ma stratégie comme je le voulais, puis sans être freiné par des membres du groupe, puis cela a été intéressant de concevoir un robot capable de jouer au puissance 4 cela montre qu'on peut concevoir des IA comme on le souhaite .

7. Bibliographie :

<https://connect4.gamesolver.org/fr/?pos=4>